



**Dashboard de Rendimiento y Análisis de KPI
de la API Restful-Booker**

Camilo Rivera, Karen Torres

Facultad de Ingeniería, Universidad de La Salle

SOF28 – Calidad en Software II

Profesor Jeferson Urrego Guerrero

27 de septiembre de 2026

Dashboard de Rendimiento y Análisis de KPI de la API Restful-Booker

Esta actividad continúa el trabajo de la Actividad 3, en la que se ejecutaron cinco perfiles de carga con k6 (load, stress, spike, soak y breakpoint) sobre el fork de Restful-Booker del equipo (Rivera y Torres, 2026). Con esa misma evidencia se construyó un dashboard que consolida los KPI de rendimiento y las métricas del contenedor, y se redactó el comentario analítico con dos KPI críticos y tres recomendaciones priorizadas. Siguiendo a Beyer et al. (2018), los indicadores medidos (SLI) se contrastan con objetivos de nivel de servicio (SLO) definidos por el equipo.

Dashboard de Rendimiento

El dashboard es una página local en HTML y JavaScript, contenida en un solo archivo y sin servidor ni dependencias externas, que cumple la función de una herramienta tipo Grafana. Los datos provienen de los resultados versionados de la Actividad 3, se consolidan en archivos CSV y un script en Python los inserta en el propio archivo del dashboard. Así, cada cifra se puede rastrear hasta su archivo de origen.

La vista reúne cuatro tarjetas KPI (latencia P95, latencia P99, throughput y tasa de error) y seis paneles: percentiles de latencia, P95 frente al SLO, throughput, CPU y RAM, Network I/O y Block I/O. Cada panel declara sus unidades, sus umbrales y su fuente, siguiendo la recomendación de Majors et al. (2022) de que un panel útil muestre pocas señales, con contexto suficiente para interpretarlas. Los umbrales son: $P95 < 800 \text{ ms}$, $P99 < 1500 \text{ ms}$, $\text{error} < 1 \%$, throughput de referencia $\geq 150 \text{ req/s}$ y un umbral de saturación de CPU del 80 %, este último como referencia y no como SLO. El panel de recursos incluye anotaciones de inicio de carga, pico de CPU y fin de carga, que permiten relacionar el comportamiento de la latencia con el uso del contenedor (Parker et al., 2020).

El dashboard, el export JSON y las capturas se encuentran en el repositorio del equipo:

<https://github.com/contracamilo/restful-booker/tree/main/master-plan/actividad-4-dashboard>.

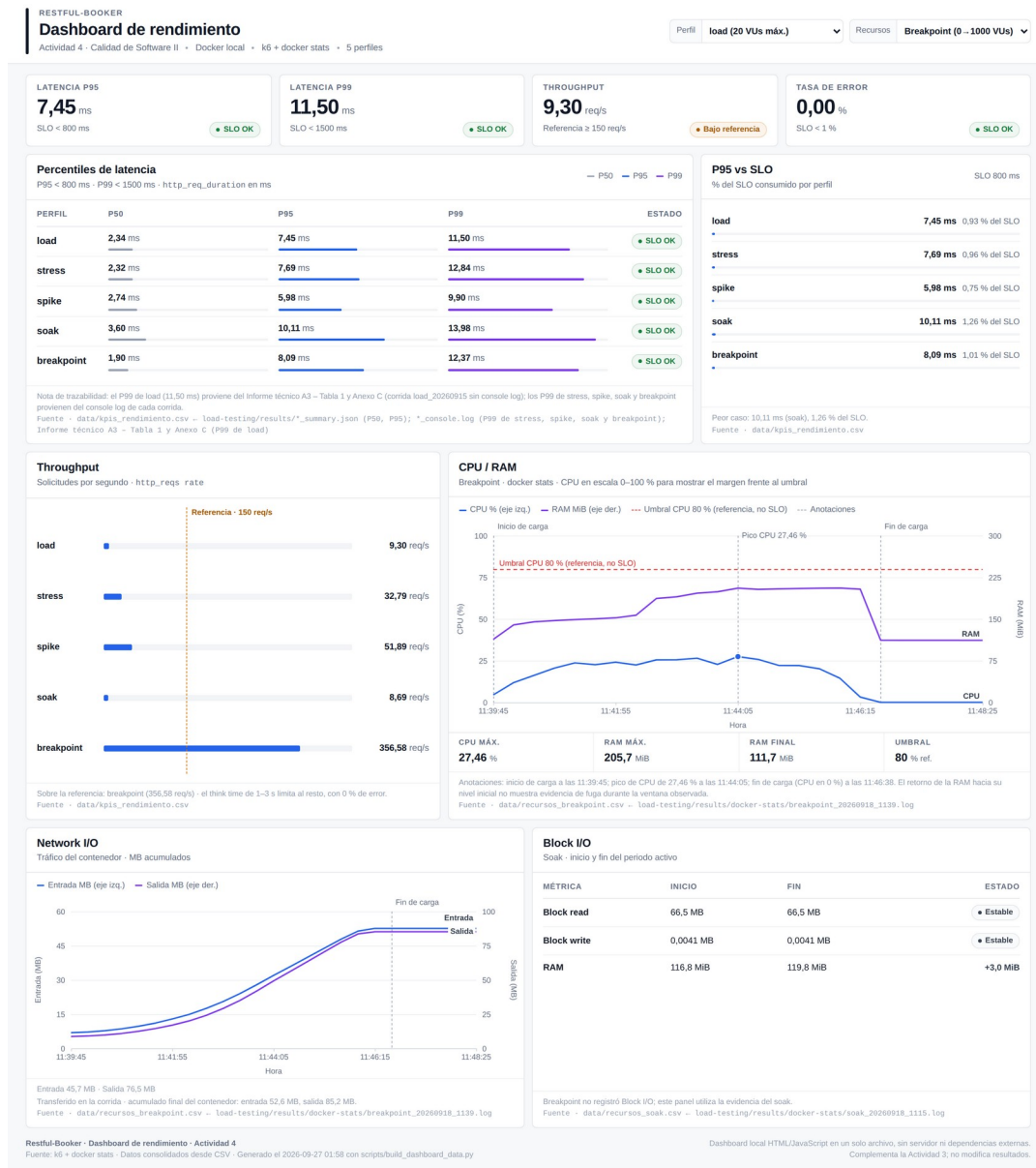


Figura 1. Vista completa del dashboard de rendimiento con el perfil load y los recursos del perfil breakpoint.

Comentario Analítico

El dashboard consolida los resultados de la Actividad 3 para los perfiles load, stress, spike, soak y breakpoint, con las métricas del contenedor registradas con docker stats. Mientras la A3 se enfocó en ejecutar y fortalecer la prueba, esta actividad analiza el sistema a partir de esa evidencia. Se seleccionaron dos KPI (indicadores clave de desempeño) críticos: la latencia P95 y el throughput, porque juntos describen la experiencia del usuario y la capacidad del servicio frente a los SLO (objetivos de nivel de servicio) definidos por el equipo.

KPI 1 — Latencia P95 (SLO < 800 ms)

Es el indicador principal: refleja el tiempo del 95 % de las solicitudes sin distorsión por valores aislados. Resultados: load 7,45 ms; stress 7,69 ms; spike 5,98 ms; soak 10,11 ms y breakpoint 8,09 ms. El peor caso (soak) consume apenas el 1,26 % del SLO. El P99 también cumple (máximo 13,98 ms frente a 1500 ms). La latencia no creció con la concurrencia: entre 15 y 1000 usuarios virtuales (VU) no se observaron señales de formación de colas en la latencia medida.



Figura 2. Percentiles P50, P95 y P99 por perfil, con su estado frente al SLO.

KPI 2 — Throughput (solicitudes por segundo, *http_reqs rate*)

Valores: load 9,30; stress 32,79; spike 51,89; soak 8,69 y breakpoint 356,58 req/s. Solo breakpoint supera la referencia de 150 req/s. La brecha en los demás perfiles se explica principalmente por el modelo de carga: el think time de 1 a 3 segundos limita la tasa de cada VU. El throughput se multiplicó casi por siete entre spike y breakpoint sin errores (0 %) ni degradación de la latencia.

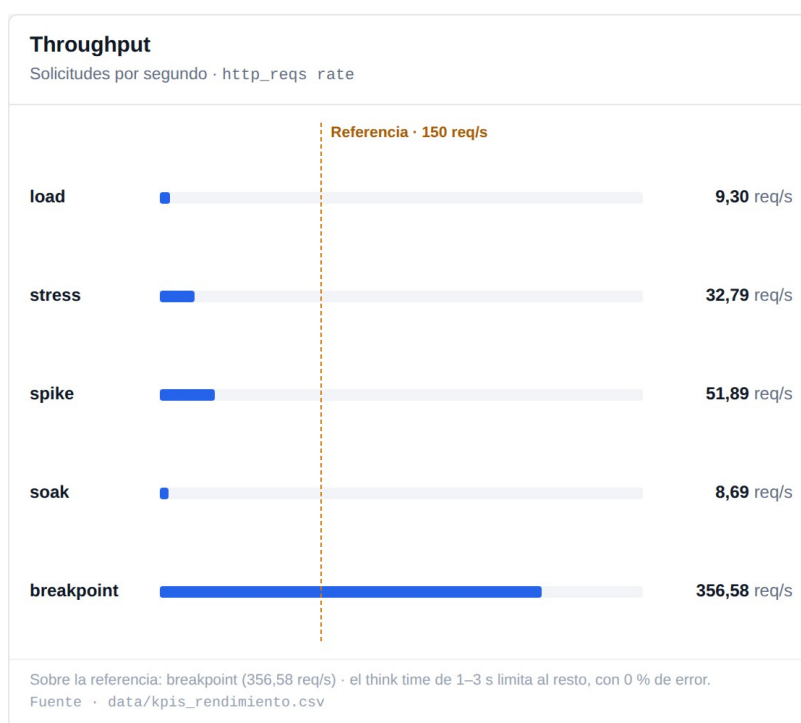


Figura 3. Throughput por perfil frente a la referencia de 150 req/s.

Métricas de Sistema

Durante breakpoint, el uso de CPU aumentó conforme avanzó la carga y alcanzó un máximo de 27,46 %, mientras que la RAM llegó a 205,7 MiB. Al finalizar la carga, ambas métricas descendieron hacia sus niveles base, sin observarse señales de saturación sostenida durante la ventana analizada; este retorno tampoco muestra evidencia de fuga de memoria. En

soak la CPU osciló entre 0,4 % y 2,9 %, la RAM entre 116,4 y 120,5 MiB, y el Block I/O no varió.

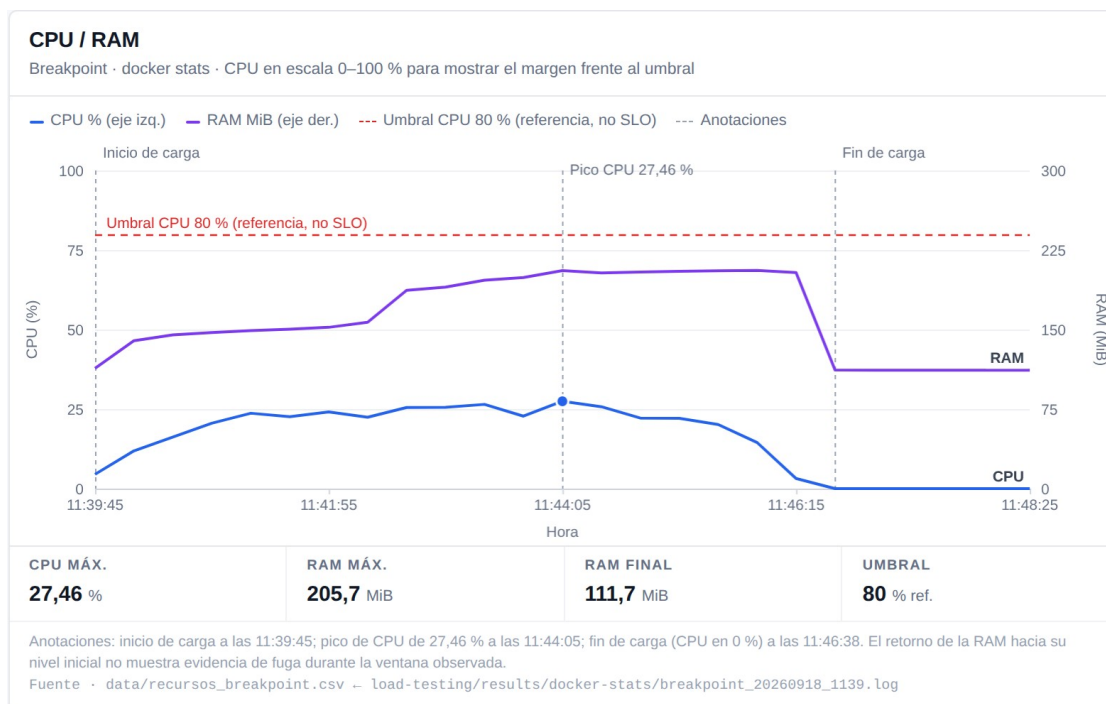


Figura 4. CPU y RAM del contenedor durante el perfil breakpoint, con el umbral de CPU del 80 % y las anotaciones de carga.

Vínculo con la Arquitectura

Restful-Booker es una aplicación Node.js/Express que corre en un solo proceso, sin módulo cluster, por lo que aprovecha un único núcleo; el 27 % de CPU sugiere margen, aunque el techo práctico lo determinaría ese proceso. La base de datos es LokiJS en memoria y la colección se crea sin índices explícitos: GET /booking usa booking.find(query) y las operaciones por identificador usan findOne({ bookingid }), que sin índices recorren la colección. Hoy son rápidas porque la colección está casi vacía, pero su costo tiende a crecer con el volumen. Además, GET /booking no implementa paginación ni límite, y el entorno probado usa un único contenedor sin réplicas ni balanceador.

Recomendaciones Priorizadas

Las recomendaciones se ordenan según el criterio de alto impacto y baja complejidad, y se orientan al sistema y no a la metodología de prueba (Tabla 1).

Tabla 1. Recomendaciones priorizadas de mejora del sistema.

Prioridad	Acción	Impacto	Esfuerzo	SLO que protege
1	Declarar índices en LokiJS: único en bookingid e índices en firstname y lastname	Alto	Bajo	P95 y P99
2	Paginar y limitar GET /booking con parámetros page y limit	Alto	Bajo	P95 y throughput
3	Escalar horizontalmente con réplicas balanceadas o módulo cluster	Alto	Alto	Throughput

La primera requiere una línea en addCollection y reduce el riesgo de que la latencia crezca con el volumen de datos. La segunda acota el tamaño de cada respuesta. La tercera queda de última porque exige antes trasladar los datos a una base compartida: con LokiJS en memoria, cada proceso tendría su propia copia de las reservas.

En conclusión, el sistema cumplió los SLO de latencia y error en el rango probado; las acciones buscan conservar ese margen cuando crezcan los datos y el tráfico.

Referencias

- Beyer, B., Murphy, N. R., Rensin, D. K., Kawahara, K., & Thorne, S. (2018). *The site reliability workbook: Practical ways to implement SRE*. O'Reilly Media.
- Majors, C., Fong-Jones, L., & Miranda, G. (2022). *Observability engineering: Achieving production excellence*. O'Reilly Media.
- Parker, A., Spoonhower, D., Mace, J., Sigelman, B., & Isaacs, R. (2020). *Distributed tracing in practice: Instrumenting, analyzing, and debugging microservices*. O'Reilly Media.
- Rivera, C., & Torres, K. (2026). *restful-booker* (versión modificada del repositorio original de M. Winteringham) [Código fuente]. GitHub. <https://github.com/contracamilo/restful-booker>

Anexo A: Capturas Adicionales del Dashboard

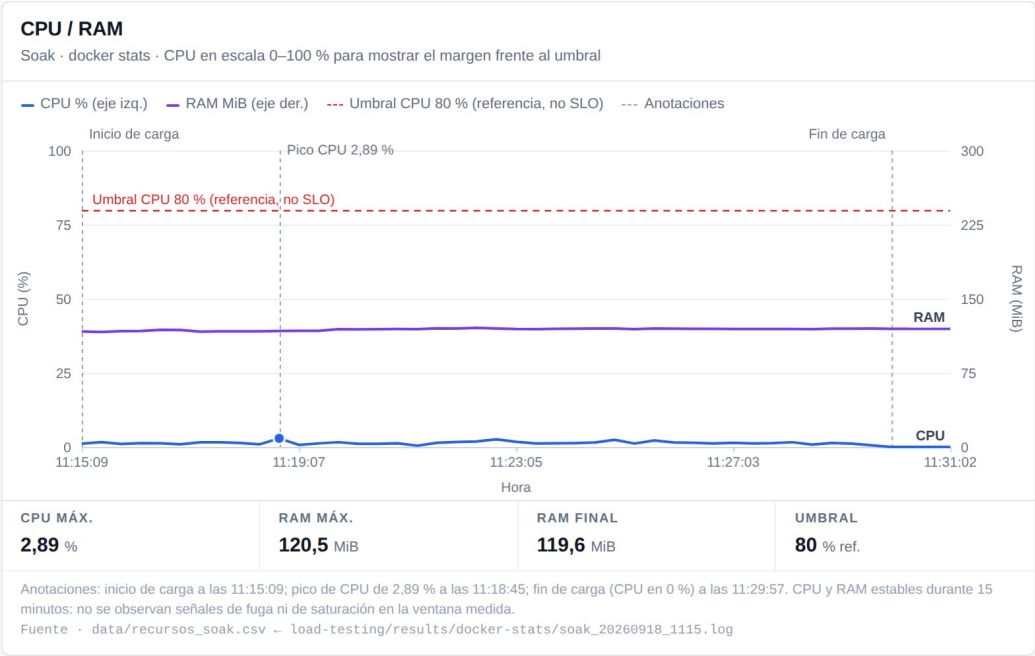


Figura A1. CPU y RAM del contenedor durante el perfil soak.

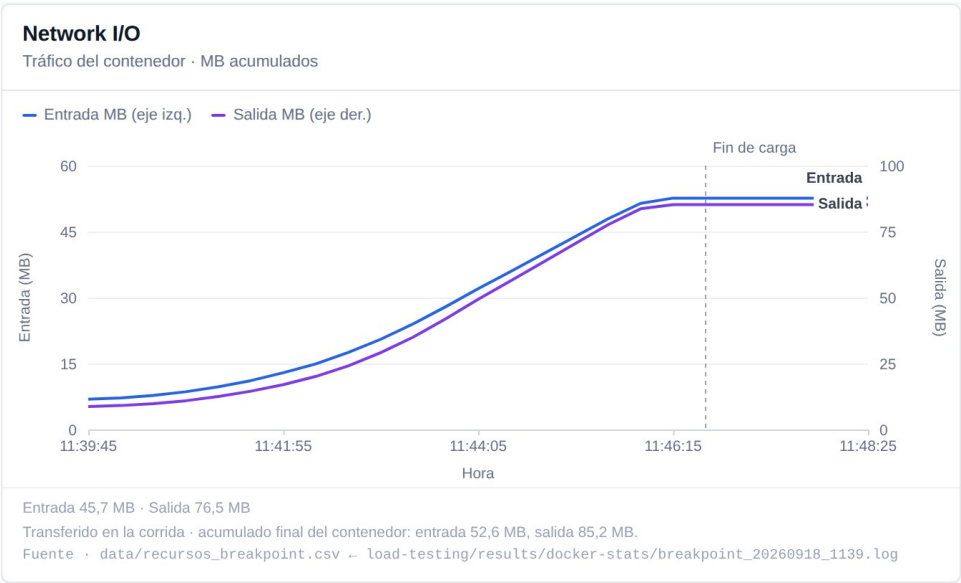


Figura A2. Tráfico de red acumulado del contenedor durante el perfil breakpoint.

Block I/O			
Soak · inicio y fin del periodo activo			
MÉTRICA	INICIO	FIN	ESTADO
Block read	66,5 MB	66,5 MB	● Estable
Block write	0,0041 MB	0,0041 MB	● Estable
RAM	116,8 MiB	119,8 MiB	+3,0 MiB
Breakpoint no registró Block I/O; este panel utiliza la evidencia del soak.			
Fuente · data/recursos_soak.csv ← load-testing/results/docker-stats/soak_20260918_1115.log			

Figura A3. Block I/O del contenedor durante el perfil soak.

Anexo B: Archivos Entregables

Junto con este informe se adjunta un archivo comprimido con el dashboard y las capturas. Los mismos archivos se encuentran en la carpeta master-plan/actividad-4-dashboard/ de la rama main del repositorio del equipo (Tabla 2).

Tabla 2. Archivos de la Actividad 4 en el repositorio del equipo.

Archivo	Contenido
dashboard/index.html	Dashboard interactivo en un solo archivo, con los datos incluidos. Se abre directamente en el navegador, sin servidor.
dashboard/dashboard_export.json	Export JSON del dashboard: paneles, fuentes, umbrales, anotaciones y limitaciones.
data/*.csv	KPI por perfil y métricas de recursos de breakpoint y soak, con la fuente de cada P99.
scripts/build_dashboard_data.py	Script que inserta en el dashboard los datos consolidados en los CSV.
capturas/*.png	Capturas del dashboard completo y de cada panel.
comentario_analitico.md	Comentario analítico incluido en este informe.
README.md	Índice técnico de la entrega y relación con la Actividad 3.

Repositorio: <https://github.com/contracamilo/restful-booker/tree/main/master-plan/actividad-4->

dashboard